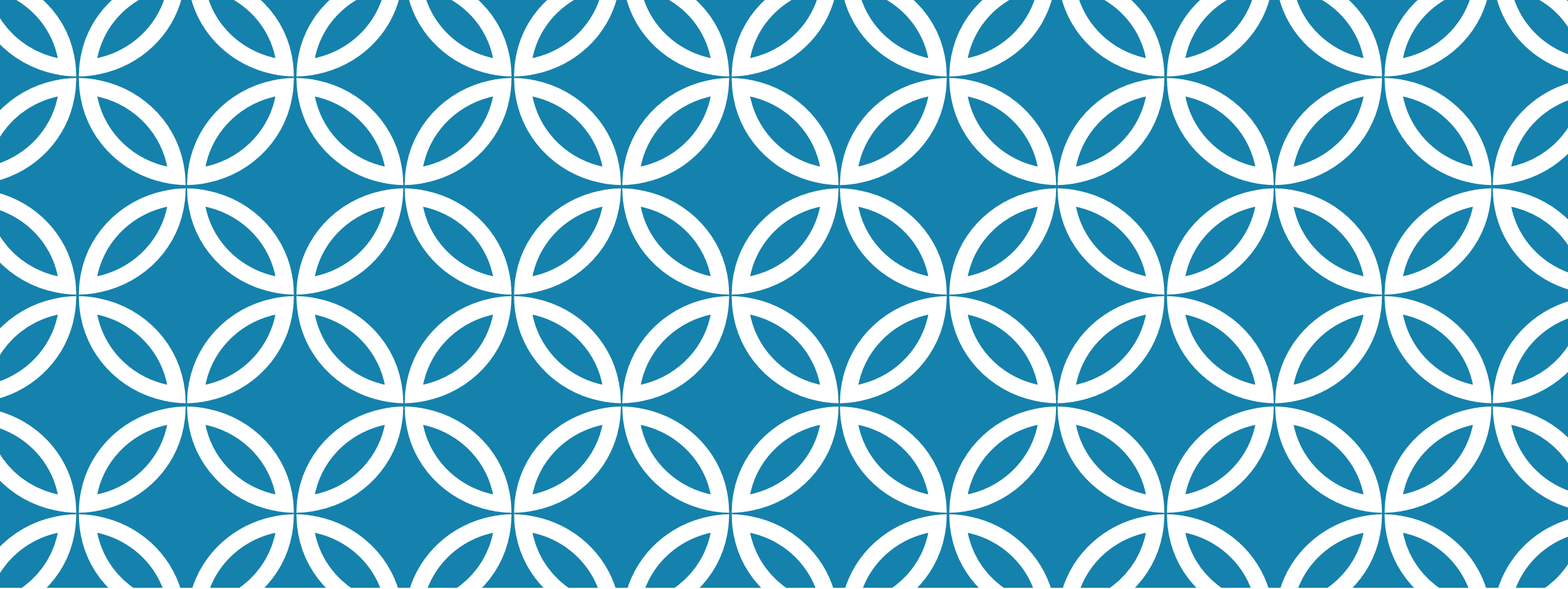




18086/8088

ARMAZENAMENTO MULTIDIMENSIONAL

Arquitetura de Computadores

ARMAZENAMENTO MULTIDIMENSIONAL

Em Assembly, não existem estruturas de dados como vetores e matrizes

- São estruturas de dados disponíveis em linguagens de alto nível

Existem sequências de valores que são armazenados em áreas contíguas e que possuem uma referência através de um nome (Rótulo)

Declaração de vetores, matrizes, ou strings não demanda diferença na sintaxe pois o resultado é sempre uma quantidade específica de bytes (ou words) contíguos

DECLARAÇÃO

.data

```
VetByte DB 100 DUP (0)      ; aloca e inicializa 100  
        ; bytes com 0 (zero)
```

```
VetWord DW 100 DUP (?)      ; aloca 100 bytes não  
        ; inicializados
```

```
MatByte DB 10 DUP (10 DUP (0)) ; aloca e inicializa  
        ; 100 bytes com 0 (zero)  
        ; área igual a VetByte
```

```
VetorPre DB 48h, 65h, 00h   ; vetor de 3 posições
```

```
String  DB 'String sem \0' ; string que ocupa 13 bytes
```

ACESSANDO ELEMENTOS DE UM VETOR

Fórmula de acesso

$$\text{Endereço_Elemento} = \text{Endereço_Base} + \text{índice} * \text{Tamanho_Elemento}$$

onde

- `Endereço_base` é o nome atribuído ao início da área de memória do vetor
- `Tamanho_Elemento` é o tamanho do elemento do vetor (bye = 1, word = 2, double-word = 4,...)
- `Índice` é o índice do elemento a ser acessado (0 a N-1 em um vetor de N elementos)

ACESSANDO ELEMENTOS DE UM VETOR

.data

```
VetByte DB 1, 2, 3 ; vetor de 3 posições
```

.code

```
mov AL, VetByte[0]; AL = 1  
mov AL, VetByte[1]; AL = 2
```

A primeira instrução atribui o valor 1 ao registrador AL e a segunda o valor 2 ao registrador AL

O modo de endereçamento é direto e a instrução ocupa 3 bytes

- Codificação da instrução primeira instrução é A0 0000r
- Codificação da instrução segunda instrução é A0 0003r

ACESSANDO ELEMENTOS DE UM VETOR

Mas o tamanho do dado deve ser levado em conta no acesso ao elemento, conforme a fórmula de acesso

```
.data
```

```
VetWord DW 1, 2, 3 ; vetor de 3 posições
```

```
.code
```

```
mov AX, VetWord[0]; AX = 1  
mov AX, VetWord[2]; AX = 2
```

As instruções utilizam AX pois o vetor é word

O valor do deslocamento do segundo elemento é 2 pois cada elemento ocupa 2 bytes

ACESSANDO ELEMENTOS DE UM VETOR

A fim de reduzir o acesso à memória por causa do endereçamento direto, podemos utilizar instruções com endereçamento indireto

- Deve-se obter o endereço inicial do vetor e atribuí-lo a um registrador de endereçamento indireto
- A instrução `lea` obtém o endereço efetivo de um nome, mas o endereço é resolvido em tempo de execução. O `offset` resolve o endereço em tempo de montagem (mais eficiente)

```
.data
```

```
VetWord DW 1, 2, 3 ; vetor de 3 posições
```

```
.code
```

```
mov BX, offset VetWord; BX = endereço de VetWord
mov AX, [BX]; AX = 1
add BX, 2
mov AX, [BX]; AX = 2
```

As instruções de atribuição ao registrador `AX` utilizam o modo de endereçamento indireto e ocupam 2 bytes, cada

ACESSANDO ELEMENTOS DE UM VETOR

Na solução anterior, cada vez que o registrador é atualizado, perde-se o valor inicial do vetor. Assim, é necessário executar a inicialização do registrador BX.

Uma solução é deixar fixo o registrador BX e utilizar os registradores de índice SI e DI

```
.data
```

```
VetWord DW 1, 2, 3 ; vetor de 3 posições
```

```
.code
```

```
mov BX, offset VetWord; BX = endereço de VetWord
xor SI, SI
mov AX, [BX][SI]; AX = 1
add SI, 2
mov AX, [BX+SI]; AX = 2
```

As instruções de atribuição ao registrador AX utilizam o modo de endereçamento indireto com índice e ocupam 2 bytes, cada

Os endereçamento `[BX][SI]` e `[BX+SI]` são idênticos e permitidos no TASM

ACESSANDO ELEMENTOS DE UM VETOR

Outra opção de acesso é através das instruções de manipulação de strings

- Somente para vetores de 1 ou 2 bytes
- Realizam a atualização do índice de forma automática

Estas instruções demandam menos bytes, mas requerem mais registradores

- LODSB (LODSW): copia elemento do vetor para AL (AX) (depende da definição do vetor)
- STOSB (STOSW): copia AL (AX) para determinada posição do vetor

As instruções trabalham com o ES

- Necessário inicializar o registrador ES no programa

ACESSANDO ELEMENTOS DE UM VETOR

```
.data
VetWord DB 1, 2, 3 ; vetor de 3 posições

.code
mov AX, @DATA
mov DS, AX
mov ES, AX
...
mov DI, offset VetWord ; DI = endereço de VetWord
cld                    ; reseta o DF → incrementar o índice

stosw                  ; [ES:DI] ← AX e DI += 2
```

A instrução `stosw` (ocupa 1 byte) copia AX para o endereço em DI e incrementa DI de 2 unidades

A instrução `cld` reseta o Direction Flag a fim de que as instruções de manipulação de string incrementem o registrador de índice (precisa fazer isso somente uma única vez no programa)

INICIALIZANDO UM VETOR

A instrução stos combinada com a de repetição rep permite inicializar uma área de memória em uma única instrução

```
.data
VetWord DB 1, 2, 3 ; vetor de 3 posições

.code
mov AX, @DATA
mov DS, AX
mov ES, AX
...
mov DI, offset VetWord ; DI = endereço de VetWord
mov CX, 3               ; Número de elementos do vetor
cld                     ; reseta o DF → incrementar o índice

mov AX, 10               ; Valor a ser atribuído ao vetor
rep stosw                ; repete 3 vezes a instrução stosw
```

O código acima atribui o valor 10 a todas as 3 posições de VetWord